SOFTWARE PROJECT LABORATORY 1 | DAY 2

# CloneScope

## Learning Manual 02

## C++17 Folder Input & Multiple C/C++ Source-File Reading

| FIELD | DETAIL |
|---|---|
| Student | Raian Huq (BSSE1708) |
| Program | Software Engineering, Institute of Information Technology, University of Dhaka |
| Supervisor | Dr. Kazi Muheymin-Us-Sakib |
| Development date | 26 October 2026 |
| Repository | github.com/raian-x8/SPL_01_BSSE1708 |
| Feature branch | feat/folder-input |
| Milestone | Basic non-recursive folder-to-multiple-file reading pipeline |

### At a glance

Day 1 already established single-file input using ifstream, getline(), user-provided paths, and basic opening-error handling. Day 2 extended that working program with C++17 filesystem operations so that a user can enter a folder and the program can discover, filter, open, and display multiple C/C++ source files.

Confirmed deliverable: Folder input, immediate-directory traversal, lowercase .c/.cpp filtering, separate opening of discovered files, and line-by-line display. The original single-file behavior remains available.

Not implemented yet: Source preprocessing, tokenization, function/block representation, clone detection, stylometry, recursive traversal, or persistent analysis results.

# 1. Learning Objectives & Concepts

By the end of Day 2, you should be able to explain both the filesystem API and the control flow of the file-input pipeline.

| CONCEPT / SYNTAX | MEANING AND WHY USED |
|---|---|
| <filesystem> | C++17 standard header containing filesystem path and directory operations. |
| namespace fs = std::filesystem; | Creates the namespace alias fs; it does not create a new library. |
| fs::path p(text) | Builds a filesystem path representation from user-entered text; constructing it does not open the file. |
| fs::is_directory(p) | Determines whether the input refers to a directory; it does not enumerate contents. |
| fs::directory_iterator(p) | Iterates entries directly inside one directory, without recursive descent or guaranteed order. |
| entry.is_regular_file() | Keeps ordinary files; skips subdirectories and non-regular entries. |
| entry.path().extension() | Extracts the extension for filtering the lowercase .c and .cpp suffixes. |
| ifstream stream(entry.path()) | Opens the current discovered file for reading; each stream is scoped to the iteration. |
| getline(stream, line) | Reads one line into a string; false at end-of-file or on read failure. |
| continue | Skips the remainder of this iteration if opening an individual file fails. |
| return 0 / return 1 | Ends main with conventional success / error status, respectively. |

## Nested loops: the core mental model

Outer for loop = one folder entry at a time. Inner while loop = one source-code line at a time within a discovered file. A new file stream is constructed for the next accepted source file.

```
Folder supplied by user
    |
    +--> Is input a directory?
           |
           +--> For each immediate entry
                    |
                    +--> Regular file AND extension .c/.cpp?
                           |
                           +--> Open file
                                  |
                                  +--> Read lines with getline()
                                  +--> Continue to next file
```

Viva detail: Output order is not guaranteed by directory_iterator. Never promise alphabetical order unless the program explicitly sorts discovered paths.

# 2. Implementation: Step by Step

## 2.1 Convert the input string into a path

```
#include <filesystem>

using namespace std;
namespace fs = std::filesystem;

// After getline(cin, filePath):
fs::path sourcePath(filePath);
```

Reason: The same sourcePath object is now usable both for directory recognition and the existing C++17 file-stream constructor.

## 2.2 Recognize a folder

```
if (fs::is_directory(sourcePath))
{
    cout << "Folder detected successfully!\n";
    // Directory-reading logic goes here.
    return 0;
}
```

When the condition is true, the program handles directory input and then exits that branch. Otherwise, execution falls through to the original ifstream inputFile(sourcePath) single-file code.

## 2.3 Visit immediate directory entries

```
for (const auto &entry : fs::directory_iterator(sourcePath))
{
    cout << entry.path() << '\n';
}
```

const auto& binds a read-only reference to each directory-entry object; auto lets the compiler infer its exact type.

## 2.4 Filter regular C/C++ files

```
if (entry.is_regular_file() &&
    (entry.path().extension() == ".c" ||
     entry.path().extension() == ".cpp"))
{
    // The selected entry is an allowed source file.
}
```

&& requires both the regular-file check and the extension group to pass. || permits either allowed extension. Current comparisons are case-sensitive: .CPP is not included.

# 3. Implementation: Open and Read

## 3.1 Open every discovered source file

```
cout << entry.path() << '\n';
ifstream sourceFile(entry.path());

if (!sourceFile.is_open())
{
    cerr << "Error: Could not open " << entry.path() << '\n';
    continue;
}

cout << "Source file opened successfully!\n";
```

Why continue? One unreadable source file should not automatically stop attempts to process the other directory entries. return would leave main(); continue moves to the next outer loop iteration.

Resource management: The ifstream destructor closes the file when its enclosing scope ends. This demonstrates RAII (Resource Acquisition Is Initialization).

## 3.2 Read the contents line by line

```
string line;
while (getline(sourceFile, line))
{
    cout << line << '\n';
}
```

The inner while continues until reading ceases to succeed. Each printed line belongs to the currently opened source file. The code currently displays source text; it does not yet save normalized tokens or clone-analysis records.

## 3.3 Preserve Day 1 single-file handling

```
// Runs when the directory condition is false.
ifstream inputFile(sourcePath);

if (!inputFile.is_open())
{
    cerr << "Error: Could not open source file.\n";
    return 1;
}

string line;
while (getline(inputFile, line))
{
    cout << line << '\n';
}
```

Backward compatibility: The pre-existing single-file flow was preserved. We verified that adding folder input did not break valid or invalid individual-file paths.

**Scope boundary**
This stage reads input files; it does not detect clones, assign tags, or make plagiarism judgments.

# 4. Verification & Testing Record

These test cases are based on successful screenshots from the development session. They are manual execution checks, not an automated test suite.

| TEST | INPUT / CONDITION | OBSERVED RESULT |
|---|---|---|
| C++17 compilation | g++ -std=c++17 src/main.cpp -o clonescope.exe | Compiled without reported errors. |
| Directory recognition | samples | Displayed folder-recognition message. |
| Directory enumeration | samples (contains hello.c) | Displayed samples\\hello.c. |
| Extension filtering | Temporary folder with student1.c, student2.cpp, notes.txt, backup/ | Included .c and .cpp; did not list .txt or subdirectory. |
| Multiple-file opening | Same temporary mixed folder | Printed successful opening messages for both source files. |
| Multiple-file reading | Same temporary mixed folder | Printed contents of both student1.c and student2.cpp. |
| Single-file regression | samples/hello.c | Successfully opened and printed original source contents. |
| Invalid-path regression | samples/nonexistent.c | Displayed expected opening error, then returned to prompt. |

## Reproduce the verified tests

```
g++ -std=c++17 src/main.cpp -o clonescope.exe
.\clonescope.exe
# Enter: samples/hello.c

.\clonescope.exe
# Enter: samples/nonexistent.c

"$env:TEMP\CloneScopeFilterTest" | .\clonescope.exe
```

Note: The mixed-content folder was created in the Windows temporary directory, outside the Git repository. Its example files contained a simple int main() { return 0; }. The exact path varies by machine.

## Tests not yet evidenced

- Empty folder and folder containing only unsupported files.
- Unreadable directory entries and filesystem exceptions.
- Nested directories, recursive scanning, file-read failures after opening.
- Uppercase extensions, deterministic ordering, multiple encodings, and large-file performance.

These are future hardening tasks; do not claim them as already validated.

# 5. Git Workflow & Actual Commits

Repository: raian-x8/SPL_01_BSSE1708. The feature branch is feat/folder-input, created from the synchronized main following the Day 1 source-file input merge.

## Meaningful Day 2 commits (verified on GitHub)

| SHORT SHA | COMMIT PURPOSE |
|---|---|
| 7cb18a8 | refactor: use C++17 filesystem path for source file input |
| ba19d68 | feat: detect directory input using C++17 filesystem |
| 441dada | feat: list directory entries using C++17 filesystem |
| ee40dd6 | feat: filter C and C++ source files during directory scan |
| 8aad25a | feat: open and read multiple C/C++ source files |

## Workflow practiced

| OPERATION | HOW IT WAS DONE |
|---|---|
| Confirm clean baseline | PowerShell: git status -sb |
| Create branch | PowerShell: git switch -c feat/folder-input |
| Implement / compile / test | VS Code editor + PowerShell g++ and executable |
| Inspect changes | VS Code Source Control diff |
| Stage files | VS Code Source Control '+' control |
| Commit changes | VS Code Source Control with descriptive messages |
| Publish / synchronize | VS Code Publish Branch / Sync Changes |
| Remote verification | Compared remote feature-branch commits and main on GitHub |

## Git concepts to remember

- Working tree: your editable files, which can differ from the last commit.
- Staging area: selected changes for the next commit.
- Commit: a local versioned snapshot with a message and SHA.
- Push/sync: uploads local commits to the corresponding remote branch.
- Pull request: proposes merging a reviewed feature branch into main.

Verified remote state: All five commits are published to feat/folder-input. They are not yet merged into main. A folder-input pull request can be created after the final input-module review and documentation commit.

# 6. Supervisor Viva Preparation

### 1. Why not use only string for paths?

std::filesystem::path gives filesystem-specific operations such as extension(), filename(), and parent_path().

### 2. Does constructing fs::path open a file?

No. It represents a path. ifstream handles file opening.

### 3. Is is_directory() the same as scanning a folder?

No. It determines whether a path refers to a directory; directory_iterator enumerates its entries.

### 4. Does directory_iterator scan recursively?

No. It visits immediate entries. Recursive traversal requires separate handling.

### 5. Why check is_regular_file()?

A directory iterator can expose subdirectories and other entries; we only want ordinary files for the current input pipeline.

### 6. Why use both && and || in filtering?

The entry must be a regular file AND its extension must be .c OR .cpp.

### 7. Why are nested loops necessary?

The outer for visits source files; the inner while reads lines of each file.

### 8. Why use continue after an opening failure?

To skip the failed file and still attempt later directory entries.

### 9. Who closes ifstream?

Its destructor automatically releases the file handle when the stream goes out of scope (RAII).

### 10. How was regression testing performed?

We reran valid and invalid single-file input tests after adding folder processing.

### 11. Does this detect plagiarism or clones?

No. The current stage only reads input. Clone candidate tags and evidence will be developed later, with human review.

### 12. What are current limitations?

Non-recursive scan, lowercase extensions only, no guaranteed file ordering, basic error handling, no stored analysis representation.

# 7. Next Milestone & Revision Checklist

## Next technical phase: preprocessing

The next development work will turn raw source text into a more useful representation for comparison. The approved midterm sequence is: preprocessing - tokenization - function/block representation - Type-1 and Type-2 clone candidate tagging - basic evidence and statistics.

## Do not overstate what exists today

| ALREADY WORKS | STILL TO BE BUILT |
|---|---|
| Read individual C/C++ source file | Robust structured representations |
| Recognize a source folder | Preprocessing and lexical tokenization |
| List immediate directory entries | Function/block extraction |
| Filter lowercase .c and .cpp | Type-1 exact/normalized matching |
| Open and display multiple files | Type-2 consistent-renaming comparison |
| Manual positive and negative tests | Evidence, statistics, CSV/JSON export |

## Before submitting the folder-input PR

- Make sure the worktree is clean and that the latest feature commits are on the remote branch.
- Review the basic input behavior, source files, and existing limitations accurately.
- Add this manual under docs/learning-notes/ and commit it separately from the source implementation.
- Create a PR with a precise summary and a test checklist; merge only after review.

## Self-check exercise (without looking at code)

Explain aloud, in under 90 seconds: 'How does CloneScope distinguish a file path from a folder path, visit the directory, filter C/C++ files, open them, and read each one safely?'

---

Closing note: Consistent project work is demonstrated by working code, reproducible tests, clear commits, and the ability to explain implementation choices. This manual records Day 2's basic folder-input milestone; it is not evidence that clone detection has been implemented.